

CRUD Operations — the foundation of everything

CRUD = Create, Read, Update, Delete. Every database operation you ever do falls into one of these 4 categories. Master these and you know 80% of MongoDB.

First, always select your database: `use mydb` — MongoDB creates it automatically when you first insert data.

C — Create (Insert)

Two methods: insert one document or many at once.

```
// Insert ONE document
db.users.insertOne({
  name: "Rahul",
  age: 25,
  city: "Indore",
  hobbies: ["cricket", "coding"]
})
// Returns: { acknowledged: true, insertedId: ObjectId("...") }
```

```
// Insert MANY documents at once
db.users.insertMany([
  { name: "Priya", age: 30, city: "Mumbai" },
  { name: "Arjun", age: 22, city: "Delhi" },
  { name: "Sneha", age: 27, city: "Pune" }
])
// Returns: { acknowledged: true, insertedIds: { 0: ..., 1: ..., 2: ... } }
```

R — Read (Find)

Fetch documents from a collection. The filter object works like "WHERE" in SQL.

```
// Get ALL documents
db.users.find()

// Get all — pretty printed (easier to read)
db.users.find().pretty()

// Get ONE document (first match)
db.users.findOne({ name: "Rahul" })

// Find with a filter (WHERE city = "Indore")
db.users.find({ city: "Indore" })

// Find and show only specific fields (projection)
// 1 = show, 0 = hide
db.users.find({}, { name: 1, city: 1, _id: 0 })

// Count documents
db.users.countDocuments({ city: "Mumbai" })

// Limit results
db.users.find().limit(5)
```

```
// Sort results (1 = ascending, -1 = descending)
db.users.find().sort({ age: 1 })

// Skip + Limit = Pagination
db.users.find().skip(10).limit(5) // page 3 of 5-per-page
```

U — Update

Two parts: (1) the filter — which document to find, (2) the update — what to change.

```
// Update ONE document — change Rahul's age
db.users.updateOne(
  { name: "Rahul" },          // filter: find this
  { $set: { age: 26 } }      // update: change this
)

// Update MANY — set all Mumbai users as verified
db.users.updateMany(
  { city: "Mumbai" },
  { $set: { isVerified: true } }
)

// Replace a document entirely (dangerous — replaces everything!)
db.users.replaceOne(
  { name: "Arjun" },
  { name: "Arjun", age: 23, city: "Delhi", updatedAt: new Date() }
)

// Upsert — update if exists, INSERT if not found
db.users.updateOne(
  { name: "Neha" },
  { $set: { age: 28, city: "Jaipur" } },
  { upsert: true }
)
```

D — Delete

```
// Delete ONE document
db.users.deleteOne({ name: "Arjun" })

// Delete MANY documents
db.users.deleteMany({ city: "Delhi" })

// Delete ALL documents in collection (careful!)
db.users.deleteMany({})

// Drop the entire collection
db.users.drop()

// Drop the entire database
db.dropDatabase()
```

Always test your filter with `find()` before running `deleteMany()` — make sure it matches what you expect!

The `_id` field

Every document automatically gets a unique `_id` field. It's an `ObjectId` — a 12-byte value that encodes a timestamp. You can also set your own `_id`.

```
// MongoDB auto-assigns _id
db.products.insertOne({ name: "Shoes" })
// { _id: ObjectId("64f1a2b3c4d5e6f7a8b9c0d1"), name: "Shoes" }

// Or set your own _id
db.products.insertOne({ _id: "prod_001", name: "Shirt", price: 599 })

// Find by _id
db.products.findOne({ _id: ObjectId("64f1a2b3c4d5e6f7a8b9c0d1") })
```

Query Operators — filter like a pro

Operators let you write powerful conditions. All operators start with `$`. Think of them as SQL's `WHERE` clause but much more expressive.

Comparison Operators

Operator	Meaning	SQL equivalent
<code>\$eq</code>	Equal to	<code>=</code>
<code>\$ne</code>	Not equal to	<code>!=</code>
<code>\$gt</code>	Greater than	<code>></code>
<code>\$gte</code>	Greater than or equal	<code>>=</code>
<code>\$lt</code>	Less than	<code><</code>
<code>\$lte</code>	Less than or equal	<code><=</code>
<code>\$in</code>	Value is in array	<code>IN (...)</code>

<code>\$nin</code>	Value NOT in array	NOT IN (...)
--------------------	--------------------	--------------

```
// age > 25
db.users.find({ age: { $gt: 25 } })

// age between 20 and 30 (inclusive)
db.users.find({ age: { $gte: 20, $lte: 30 } })

// city is either Mumbai or Delhi
db.users.find({ city: { $in: ["Mumbai", "Delhi"] } })

// city is NOT Indore
db.users.find({ city: { $ne: "Indore" } })

// age is not 25 or 30
db.users.find({ age: { $nin: [25, 30] } })
```

Logical Operators

```
// $and - age > 20 AND city is Mumbai
db.users.find({
  $and: [
    { age: { $gt: 20 } },
    { city: "Mumbai" }
  ]
})

// Shorthand (implicit AND) - same result:
db.users.find({ age: { $gt: 20 }, city: "Mumbai" })

// $or - city is Mumbai OR Delhi
db.users.find({
  $or: [
    { city: "Mumbai" },
    { city: "Delhi" }
  ]
})

// $nor - neither Mumbai nor Delhi
db.users.find({
  $nor: [
    { city: "Mumbai" },
    { city: "Delhi" }
  ]
})

// $not - age is NOT greater than 25
db.users.find({ age: { $not: { $gt: 25 } } })
```

Element Operators

```
// $exists - documents that HAVE the "phone" field
db.users.find({ phone: { $exists: true } })
```

```
// Documents that DON'T have "phone" field
db.users.find({ phone: { $exists: false } })

// $type - find documents where age is stored as a number
db.users.find({ age: { $type: "number" } })
```

Array Operators

```
// $all - hobbies array must contain BOTH "cricket" AND "coding"
db.users.find({ hobbies: { $all: ["cricket", "coding"] } })

// $size - hobbies array has exactly 3 items
db.users.find({ hobbies: { $size: 3 } })

// $elemMatch - find users with a score object > 80 in subjects array
db.students.find({
  scores: { $elemMatch: { score: { $gt: 80 } } }
})
```

String / Regex Search

```
// Find names starting with "R" (case-insensitive)
db.users.find({ name: { $regex: /^R/i } })

// Find emails containing "gmail"
db.users.find({ email: { $regex: "gmail" } })

// Text search (requires a text index - covered in section 4)
db.articles.find({ $text: { $search: "mongodb tutorial" } })
```

Update Operators + Schema Design

Update Operators — modify specific fields

Operator	What it does
\$set	Set a field value (creates if doesn't exist)
\$unset	Remove a field from document
\$inc	Increment/decrement a number
\$mul	Multiply a number

<code>\$rename</code>	Rename a field
<code>\$push</code>	Add item to an array
<code>\$pull</code>	Remove item from array by value
<code>\$addToSet</code>	Add to array only if not already present
<code>\$pop</code>	Remove first (-1) or last (1) item from array
<code>\$min / \$max</code>	Update only if new value is lower/higher
<code>\$currentDate</code>	Set field to current date/timestamp

```
// $set - add/update fields
db.users.updateOne(
  { name: "Rahul" },
  { $set: { age: 26, city: "Bhopal", updatedAt: new Date() } }
)

// $unset - remove a field
db.users.updateOne({ name: "Rahul" }, { $unset: { age: "" } })

// $inc - increment likes by 1 (great for counters)
db.posts.updateOne({ _id: postId }, { $inc: { likes: 1 } })

// $inc - decrement stock by 2
db.products.updateOne({ _id: prodId }, { $inc: { stock: -2 } })

// $push - add a hobby to the array
db.users.updateOne(
  { name: "Rahul" },
  { $push: { hobbies: "photography" } }
)

// $push with $each - add multiple items at once
db.users.updateOne(
  { name: "Rahul" },
  { $push: { hobbies: { $each: ["cooking", "travel"] } } }
)

// $pull - remove "cricket" from hobbies
db.users.updateOne(
  { name: "Rahul" },
  { $pull: { hobbies: "cricket" } }
)
```

```

)

// $addToSet - add tag only if it doesn't exist (no duplicates)
db.posts.updateOne(
  { _id: postId },
  { $addToSet: { tags: "mongodb" } }
)

// $rename - rename field "phone" to "mobile"
db.users.updateMany({}, { $rename: { "phone": "mobile" } })

// $currentDate - set lastLogin to now
db.users.updateOne(
  { name: "Rahul" },
  { $currentDate: { lastLogin: true } }
)

```

Schema Design — the most important concept

MongoDB is schema-free but you **MUST** plan your data structure. The big question is always: embed vs reference?

Approach 1 — Embedded Documents (denormalization)

Store related data inside the same document. Like nesting objects.

```

// User with embedded address and orders
{
  _id: ObjectId("..."),
  name: "Rahul",
  email: "rahul@example.com",
  address: {                                // embedded object
    street: "MG Road",
    city: "Indore",
    pincode: 452001
  },
  recentOrders: [                          // embedded array
    { item: "Shoes", price: 999, date: "2024-01-10" },
    { item: "Shirt", price: 499, date: "2024-01-15" }
  ]
}

```

Use when

Data is always fetched together. One-to-one or one-to-few relationships. e.g. user + address, post + comments (few)

Avoid when

Embedded data grows unboundedly. Same sub-document needs to be queried independently. e.g. don't embed 10,000 orders inside a user

Approach 2 — References (normalization)

Store the `_id` of a related document — like a foreign key in SQL.

```
// User document
{ _id: ObjectId("user_001"), name: "Rahul", email: "rahul@example.com" }

// Order document - references the user
{
  _id: ObjectId("order_001"),
  userId: ObjectId("user_001"), // reference to user!
  item: "Shoes",
  price: 999,
  date: "2024-01-10"
}

// To get user + their orders, run two queries:
const user = db.users.findOne({ name: "Rahul" })
const orders = db.orders.find({ userId: user._id }).toArray()
```

Use when

Data is large or grows a lot. Documents are queried independently. Many-to-many relationships. e.g. users and orders

Trade-off

Needs multiple queries (no JOINS by default — use `$lookup` in aggregation to JOIN across collections)

Golden rule — model data the way your app uses it, not the way it "logically exists". If you always fetch A with B together, embed B inside A.

Indexes — the secret to performance

Without an index, MongoDB scans EVERY document to find matches (called a collection scan). With an index it jumps straight to the result. Think of it like an index in a book — instead of reading every page, you look up the page number.

Without indexes, queries on large collections (millions of docs) can take seconds. With the right index, same query takes milliseconds.

1. Single Field Index

```
// Create an index on the "email" field
db.users.createIndex({ email: 1 })
// 1 = ascending, -1 = descending
```

```
// Create index on "age" descending
db.users.createIndex({ age: -1 })

// View all indexes on a collection
db.users.getIndexes()

// Drop an index
db.users.dropIndex({ email: 1 })

// _id always has an index automatically – you never need to create it
```

2. Unique Index

Ensures no two documents have the same value for a field. Essential for emails, usernames, etc.

```
// Email must be unique across all users
db.users.createIndex({ email: 1 }, { unique: true })

// Now this will throw a duplicate key error:
db.users.insertOne({ email: "rahul@example.com" }) // if already exists
```

3. Compound Index

Index on multiple fields together. Used when your queries filter by more than one field.

```
// Index on city + age together
db.users.createIndex({ city: 1, age: -1 })

// This query now uses the compound index (fast):
db.users.find({ city: "Mumbai", age: { $gt: 25 } })

// Rule: queries must start from the LEFT field of the compound index
// city + age    ✓ uses index
// city only    ✓ uses index (partial)
// age only     ✗ does NOT use compound index
```

4. Text Index

For full-text search inside string fields. Only one text index per collection.

```
// Create text index on "title" and "body" fields
db.articles.createIndex({ title: "text", body: "text" })

// Now search for articles containing these words
db.articles.find({ $text: { $search: "mongodb performance" } })

// Search for exact phrase
db.articles.find({ $text: { $search: "\"mongodb tutorial\"" } })

// Exclude a word (articles with "mongodb" but NOT "beginner")
db.articles.find({ $text: { $search: "mongodb -beginner" } })
```

```
// Sort by relevance score
db.articles.find(
  { $text: { $search: "mongodb" } },
  { score: { $meta: "textScore" } }
).sort({ score: { $meta: "textScore" } })
```

5. TTL Index — auto-delete documents

Documents automatically delete themselves after N seconds. Perfect for sessions, OTPs, cache.

```
// Sessions expire after 30 minutes (1800 seconds)
db.sessions.createIndex(
  { createdAt: 1 },
  { expireAfterSeconds: 1800 }
)

// Insert a session — it auto-deletes after 30 min
db.sessions.insertOne({
  userId: "user_001",
  token: "abc123",
  createdAt: new Date()
})
```

6. explain() — see if your index is being used

```
// Check if a query uses an index
db.users.find({ email: "rahul@example.com" }).explain("executionStats")

// Look for:
// "stage": "IXSCAN" ← index scan (good, fast!)
// "stage": "COLLSCAN" ← collection scan (bad, slow!)
```

Create indexes for every field you query or sort by frequently. But don't over-index — each index slows down writes slightly and uses disk space.

Aggregation Pipeline — MongoDB's superpower

Aggregation lets you transform, group, and analyze data in stages. Think of it as an assembly line — data goes in, passes through stages, transformed data comes out. Each stage's output becomes the next stage's input.

```
db.collection.aggregate([
  { stage1 },
  { stage2 },
  { stage3 },
  // ...
])
```

The Core Stages

Stage	What it does	SQL equivalent
<code>\$match</code>	Filter documents	WHERE
<code>\$group</code>	Group + aggregate values	GROUP BY
<code>\$sort</code>	Sort documents	ORDER BY
<code>\$project</code>	Select/reshape fields	SELECT columns
<code>\$limit</code>	Take only N documents	LIMIT
<code>\$skip</code>	Skip N documents	OFFSET
<code>\$lookup</code>	Join another collection	JOIN
<code>\$unwind</code>	Flatten an array field	—
<code>\$addFields</code>	Add computed fields	computed columns
<code>\$count</code>	Count documents	COUNT(*)

`$match` — filter first (always put early for performance)

```
db.orders.aggregate([
  { $match: { status: "delivered", amount: { $gt: 500 } } }
])
```

`$group` — the most powerful stage

```
// Count total orders per city
db.orders.aggregate([
  {
    $group: {
      _id: "$city", // group BY city
      totalOrders: { $sum: 1 }, // count each doc
      totalRevenue: { $sum: "$amount" },
      avgAmount: { $avg: "$amount" },
    }
  }
])
```



```

        totalRevenue: { $sum: "$amount" },
        orderCount: { $sum: 1 }
    }},
    { $sort: { totalRevenue: -1 } },           // stage 3: sort desc
    { $limit: 3 },                             // stage 4: top 3 only
    { $project: {                             // stage 5: clean output
        city: "$_id",
        totalRevenue: 1,
        orderCount: 1,
        _id: 0
    }}
])

```

Mongoose — MongoDB + Node.js

Mongoose is an ODM (Object Document Mapper) — it's a layer on top of MongoDB that adds structure, validation, and helper methods. It's what you use in real Node.js/Express apps.

Install

```
npm install mongoose
```

1. Connect to MongoDB

```

const mongoose = require("mongoose")

mongoose.connect("mongodb://127.0.0.1:27017/myapp")
  .then(() => console.log("MongoDB connected!"))
  .catch(err => console.log("Error:", err))

```

2. Define a Schema

A Schema defines the structure and validation rules for your documents. This is the key difference from raw MongoDB — you get structure.

```

const { Schema, model } = require("mongoose")

const userSchema = new Schema({
  name:      { type: String,   required: true, trim: true },
  email:     { type: String,   required: true, unique: true, lowercase: true },
  age:       { type: Number,   min: 0, max: 120 },
  city:      { type: String,   default: "Unknown" },
  hobbies:   [String],        // array of strings
  isActive:  { type: Boolean,  default: true },
  createdAt: { type: Date,     default: Date.now },
  address:   {                 // nested object
    street: String,
    pincode: Number
  }
})

// Schema types: String, Number, Boolean, Date, Array,

```

```
// ObjectId, Mixed, Buffer, Map
```

3. Create a Model

A model is a class built from a schema. It represents the collection and gives you all the query methods.

```
const User = model("User", userSchema)

// Mongoose auto-creates collection named "users" (lowercase + plural)
```

4. CRUD with Mongoose

```
// CREATE
const newUser = new User({ name: "Rahul", email: "rahul@example.com", age: 25 })
await newUser.save()

// Shorthand create:
await User.create({ name: "Priya", email: "priya@example.com", age: 30 })

// READ
const allUsers = await User.find()
const oneUser = await User.findById("64f1a2b3...")
const byEmail = await User.findOne({ email: "rahul@example.com" })
const mumbaiUsers = await User.find({ city: "Mumbai" }).sort({ age: 1 }).limit(10)

// UPDATE
await User.findByIdAndUpdate(id, { $set: { age: 26 } }, { new: true })
// { new: true } returns the updated doc instead of the old one

await User.updateMany({ city: "Delhi" }, { $set: { zone: "North" } })

// DELETE
await User.findByIdAndDelete(id)

await User.deleteMany({ isActive: false })
```

5. Schema Validation

```
// Mongoose validates before saving – this will throw an error:
const badUser = new User({ age: -5 }) // required "name" missing, age below min
await badUser.save() // throws ValidationError!

// Custom validator
const productSchema = new Schema({
  price: {
    type: Number,
    validate: {
      validator: v => v > 0,
      message: "Price must be positive"
    }
  }
})
```

6. Virtuals — computed fields

```
// Virtual field – not stored in DB, computed on the fly
userSchema.virtual("fullName").get(function() {
  return `${this.firstName} ${this.lastName}`
})

// user.fullName → "Rahul Sharma" (not in MongoDB, computed in JS)
```

7. Middleware (Hooks)

Run code automatically before/after certain operations.

```
// Hash password before saving (pre-hook)
userSchema.pre("save", async function(next) {
  if (this.isModified("password")) {
    this.password = await bcrypt.hash(this.password, 10)
  }
  next()
})

// Log after a user is deleted (post-hook)
userSchema.post("findOneAndDelete", function(doc) {
  console.log(`User ${doc.name} was deleted`)
})
```

8. Relationships with populate()

Mongoose's version of \$lookup — it automatically fetches the referenced document.

```
// Post schema references User by ObjectId
const postSchema = new Schema({
  title: String,
  body: String,
  author: { type: Schema.Types.ObjectId, ref: "User" } // reference!
})
const Post = model("Post", postSchema)

// Create post with reference
await Post.create({ title: "Hello MongoDB", author: userId })

// Populate – fetch the actual user object when querying posts
const posts = await Post.find().populate("author", "name email")

// Each post now has: { title: "...", author: { name: "Rahul", email: "..." } }
```

The `.populate()` call does a second query behind the scenes. For heavy production use, consider \$lookup in aggregation instead — it's more efficient.

Complete app structure (Express + Mongoose)

```
// app.js – minimal Express + Mongoose setup
const express = require("express")
const mongoose = require("mongoose")
const app = express()

app.use(express.json())
```

```
mongoose.connect("mongodb://127.0.0.1:27017/myapp")

// Routes
app.get("/users", async (req, res) => {
  const users = await User.find()
  res.json(users)
})
app.post("/users", async (req, res) => {
  const user = await User.create(req.body)
  res.status(201).json(user)
})
app.put("/users/:id", async (req, res) => {
  const user = await User.findByIdAndUpdate(req.params.id, req.body, { new: true })
  res.json(user)
})
app.delete("/users/:id", async (req, res) => {
  await User.findByIdAndDelete(req.params.id)
  res.json({ message: "Deleted" })
})

app.listen(3000, () => console.log("Server running on port 3000"))
```